

```
#include <stdio.h>
```

```
void merge(int arr[], int left, int mid, int right) {
```

```
    int n1 = mid - left + 1;
```

```
    int n2 = right - mid;
```

```
    int L[1000000], R[1000000];
```

```
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
```

```
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
```

```
    int i = 0, j = 0, k = left;
```

```
    while (i < n1 && j < n2) {
```

```
        if (L[i] <= R[j]) arr[k++] = L[i++];
```

```
        else arr[k++] = R[j++];
```

```
    }
```

```
    while (i < n1) arr[k++] = L[i++];
```

```
    while (j < n2) arr[k++] = R[j++];
```

```
}
```

```
void mergeSort(int arr[], int left, int right) {
```

```
    if (left < right) {
```

```
        int mid = (left + right) / 2;
```

```
mergeSort(arr, left, mid);  
mergeSort(arr, mid + 1, right);  
merge(arr, left, mid, right);  
}  
}
```

```
int main() {  
    int n;  
    scanf("%d", &n);  
  
    int marks[100000];  
    for (int i = 0; i < n; i++) scanf("%d", &marks[i]);  
  
    mergeSort(marks, 0, n - 1);  
  
    for (int i = 0; i < n; i++) printf("%d ", marks[i]);  
    printf("\n");  
  
    return 0;  
}
```

```
#include <stdio.h>
```

```
void merge(int arr[], int left, int mid, int right) {
```

```
    int n1 = mid - left + 1;
```

```
    int n2 = right - mid;
```

```
    int L[1000000], R[1000000];
```

```
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
```

```
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
```

```
    int i = 0, j = 0, k = left;
```

```
    while (i < n1 && j < n2) {
```

```
        if (L[i] <= R[j]) arr[k++] = L[i++];
```

```
        else arr[k++] = R[j++];
```

```
    }
```

```
    while (i < n1) arr[k++] = L[i++];
```

```
    while (j < n2) arr[k++] = R[j++];
```

```
    }
```

```
void mergeSort(int arr[], int left, int right) {
```

```
    if (left < right) {
```

```
        int mid = (left + right) / 2;
```

```
mergeSort(arr, left, mid);  
mergeSort(arr, mid + 1, right);  
merge(arr, left, mid, right);  
}  
}
```

```
int main() {  
    int n;  
    scanf("%d", &n);  
  
    int price[1000000];  
    for (int i = 0; i < n; i++) scanf("%d", &price[i]);  
  
    mergeSort(price, 0, n - 1);  
  
    for (int i = 0; i < n; i++) printf("%d ", price[i]);  
    printf("\n");  
  
    return 0;  
}
```

```
#include <stdio.h>
```

```
void merge(int arr[], int left, int mid, int right) {  
    int n1 = mid - left + 1;  
    int n2 = right - mid;
```

```
    int L[1000000], R[1000000];
```

```
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];  
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
```

```
    int i = 0, j = 0, k = left;
```

```
    while (i < n1 && j < n2) {  
        if (L[i] <= R[j]) arr[k++] = L[i++];  
        else arr[k++] = R[j++];  
    }
```

```
    while (i < n1) arr[k++] = L[i++];  
    while (j < n2) arr[k++] = R[j++];  
}
```

```
void mergeSort(int arr[], int left, int right) {  
    if (left < right) {  
        int mid = (left + right) / 2;
```

```
mergeSort(arr, left, mid);  
mergeSort(arr, mid + 1, right);  
merge(arr, left, mid, right);  
}  
}
```

```
int main() {  
    int n;  
    scanf("%d", &n);
```

```
    int salary[100000];  
    for (int i = 0; i < n; i++) scanf("%d", &salary[i]);
```

```
    mergeSort(salary, 0, n - 1);
```

```
    for (int i = 0; i < n; i++) printf("%d ", salary[i]);  
    printf("\n");
```

```
    printf("Lowest Salary: %d\n", salary[0]);  
    printf("Highest Salary: %d\n", salary[n - 1]);  
    printf("Difference: %d\n", salary[n - 1] - salary[0]);
```

```
    return 0;  
}
```

```
#include <stdio.h>
```

```
void swap(int *a, int *b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

```
int partition(int arr[], int low, int high) {  
    int pivot = arr[high];  
    int i = low - 1;
```

```
    for (int j = low; j < high; j++) {  
        if (arr[j] < pivot) {  
            i++;  
            swap(&arr[i], &arr[j]);  
        }  
    }
```

```
    swap(&arr[i + 1], &arr[high]);  
    return i + 1;  
}
```

```
void quickSort(int arr[], int low, int high) {  
    if (low < high) {
```

```
int pi = partition(arr, low, high);  
quickSort(arr, low, pi - 1);  
quickSort(arr, pi + 1, high);  
}  
}
```

```
int main() {  
int n;  
scanf("%d", &n);
```

```
int time[100000];  
for (int i = 0; i < n; i++) scanf("%d", &time[i]);
```

```
quickSort(time, 0, n - 1);
```

```
for (int i = 0; i < n; i++) printf("%d ", time[i]);  
printf("\n");
```

```
return 0;  
}
```

```
#include <stdio.h>
```

```
void swap(int *a, int *b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

```
int partition(int arr[], int low, int high) {  
    int pivot = arr[high];  
    int i = low - 1;
```

```
    for (int j = low; j < high; j++) {  
        if (arr[j] < pivot) {  
            i++;  
            swap(&arr[i], &arr[j]);  
        }  
    }
```

```
    swap(&arr[i + 1], &arr[high]);  
    return i + 1;  
}
```

```
void quickSort(int arr[], int low, int high) {  
    if (low < high) {
```

```
int pi = partition(arr, low, high);  
quickSort(arr, low, pi - 1);  
quickSort(arr, pi + 1, high);  
}  
}
```

```
int main() {  
int n;  
scanf("%d", &n);  
  
int id[1000000];  
for (int i = 0; i < n; i++) scanf("%d", &id[i]);  
  
quickSort(id, 0, n - 1);  
  
for (int i = 0; i < n; i++) printf("%d ", id[i]);  
printf("\n");  
  
return 0;  
}
```

```
#include <stdio.h>
```

```
void swap(int *a, int *b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

```
int partition(int arr[], int low, int high) {  
    int pivot = arr[high];  
    int i = low - 1;
```

```
    for (int j = low; j < high; j++) {  
        if (arr[j] > pivot) {  
            i++;  
            swap(&arr[i], &arr[j]);  
        }  
    }
```

```
    swap(&arr[i + 1], &arr[high]);  
    return i + 1;  
}
```

```
void quickSort(int arr[], int low, int high) {  
    if (low < high) {
```

```
int pi = partition(arr, low, high);  
quickSort(arr, low, pi - 1);  
quickSort(arr, pi + 1, high);  
}  
}
```

```
int main() {  
int n;  
scanf("%d", &n);
```

```
int sales[1000000];  
for (int i = 0; i < n; i++) scanf("%d", &sales[i]);
```

```
quickSort(sales, 0, n - 1);
```

```
for (int i = 0; i < n; i++) printf("%d ", sales[i]);  
printf("\n");
```

```
int average = (sales[0] + sales[1] + sales[2]) / 3;
```

```
printf("Top 3: %d %d %d\n", sales[0], sales[1],  
sales[2]);
```

```
printf("Average: %d\n", average);
```

```
return 0;
```